



Architecting MLOps Solutions

LECTURE

Recommended MLOps Architectures



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

The lecture **Recommended MLOps Architectures** explores the importance, dimensions, solutions, and infrastructure setup leading to a complete MLOps architecture.

Importance of MLOps Architecture

Setting an ML project up for success starts with architecture

Simplicity

When ML projects are well architected, the downstream management, maintenance, and monitoring of the project is simplified.

Efficiency

When ML projects are well architected, processes around the project become more efficient.

Scalability

When ML projects are well architected, they can easily be scaled to adapt to changing requirements for infrastructure and compute.

Collaboration

When ML projects are well architected, it's easy for different users and different types of users to collaborate effectively.



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Choosing the right MLOps architecture is essential to ensure project infrastructure is optimized for performance and ease of use. A well-designed architecture simplifies management, maintenance, and monitoring of ML projects, reducing complexity and improving user experience. It drives operational efficiency by lowering costs, accelerating timelines, and enabling faster improvements through CI/CD workflows. Scalability is built in, allowing systems to adapt to changing demands without major rework, while preventing future issues by incorporating scalable DataOps capabilities early on. Additionally, it facilitates collaboration by providing a familiar structure that supports smooth handovers, continuity, and quicker onboarding for new team members.

Importance of MLOps Architecture

Setting an ML project up for success starts with architecture

Simplicity

When ML projects are well architected, the downstream management, maintenance, and monitoring of the project is simplified.

Efficiency

When ML projects are well architected, processes around the project become more efficient.

Scalability

When ML projects are well architected, they can easily be scaled to adapt to changing requirements for infrastructure and compute.

Collaboration

When ML projects are well architected, it's easy for different users and different types of users to collaborate effectively.



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

- **Strategic Approach:**
 - **Incorporate best practices** in **DataOps**, **DevOps**, and **ModelOps** to create a cohesive and scalable framework.
 - **Use tools** like Apache Spark for data processing, Delta Lake for data storage, and Unity Catalog for data governance to support the architecture.
 - **Implement workflows** that promote easy integration and continuous delivery, enhancing both productivity and project quality.

Dimensions of Architecture

Differentiating initial organization/setup and ongoing workflows

Infrastructure

The organization, governance, and setup of environments, data, compute and other resources.

- Set up one time (per project or per team/organization)
- Crucial to downstream success of project(s)

Workflow

The processes that ML practitioners follow within a defined architecture to achieve success on an ML project.

- Repeatable, fluid processes specific to a project
- Aligned to organizational best practices



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

- **Critical components** of architecture in MLOps = **Infrastructure** and **Workflow**.
 - Each plays a unique role in setting up and operating machine learning projects effectively.
 - Infrastructure provides the necessary resources and framework, while workflows dictate the efficient use of these resources in day-to-day operations.
- **Infrastructure: Typically set up once per project or organization**
 - **Provides a stable and scalable foundation** that supports all machine learning activities
 - Involves the organization, governance, and setup of environments, data, compute resources, and other necessary infrastructure components.
 - **Crucial for downstream success**: A well-planned infrastructure simplifies ongoing operations and reduces the complexity of maintenance and scaling.

Dimensions of Architecture

Differentiating initial organization/setup and ongoing workflows

Infrastructure

The organization, governance, and setup of environments, data, compute and other resources.

- Set up one time (per project or per team/organization)
- Crucial to downstream success of project(s)

Workflow

The processes that ML practitioners follow within a defined architecture to achieve success on an ML project.

- Repeatable, fluid processes specific to a project
- Aligned to organizational best practices

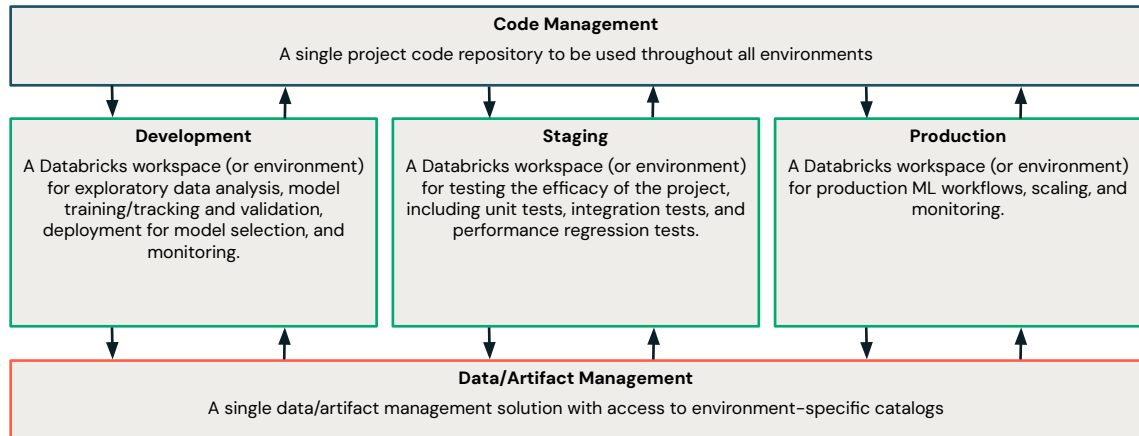


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

- **Workflow:**
 - Enables **efficient execution of machine learning projects** by ensuring that all activities are aligned with both project goals and organizational standards.
 - Consists of the **processes within the defined architecture** to drive project success.
 - **Characteristics:**
 - **Repeatable and fluid:** Workflows are designed to be project-specific yet adhere to organizational best practices.
 - Supports the **automation and scheduling** of tasks to maintain or enhance machine learning models efficiently.
- **Best Practices:**
 - **Develop infrastructure with scalability and flexibility in mind** to accommodate future growth and technological advancements.
 - Design **workflows that can easily adapt to changes in the project scope or data** without extensive reconfigurations.

Recommended MLOps Architecture

A high-level view of code, data, and ML environments



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

The Databricks MLOps architecture is built around key components that ensure consistency, scalability, and efficiency. Code management relies on a single project repository, maintaining control and uniformity across changes and deployments. Environments are clearly defined: development supports experimentation, exploratory analysis, and initial validation; staging acts as the testing ground to confirm model readiness for production; and production handles deployment, scaling, and continuous monitoring to sustain performance. Data and artifact management is unified across all environments, preserving integrity and accessibility while enabling smooth transitions throughout the lifecycle. This architecture delivers a seamless workflow from development to production, reduces disruptions, and provides a scalable framework capable of handling growing workloads and data complexity.

MLOps Solution

Infrastructure through production



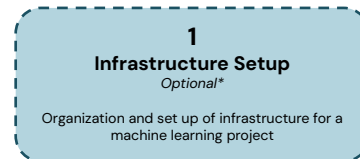
© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

- **4 Phases of the MLOps Solution:**
- **1. Infrastructure Setup (Optional):**
 - setup typically requires **one-time configuration**
 - Organization and setup of infrastructure for a machine learning project.
 - preliminary step, which may be optional if suitable infrastructure already exists. This but is crucial for scaling and security purposes.
- **Cyclical = Continuous Improvement & Model Refreshing**
 - **2. Development:** Data scientists and ML engineers **collaborate** to create robust ML models by processing and analyzing data, building and training models.
 - **3. Staging:** Includes **comprehensive testing** of the ML pipelines to ensure accuracy, efficiency, and readiness for production.
 - Staging acts as a quality gate that models must pass before deployment.
 - **4. Production:** Deploying the ML solution in a production environment where it is scaled, monitored, and maintained to meet ongoing business requirements.

Infrastructure Setup

Getting set up for a machine learning project

- **Who:** Architect or Engineer
- **How often:** Set up once
- **What:**
 - A Unity Catalog metastore
 - One or three Databricks workspaces
 - Three data catalogs: dev, staging, prod
 - Command line environment
 - *MLOps Stacks* project (per project)
 - Git repository (per project)
- **How:**
 - Manual setup/Terraform
 - MLOps Stacks



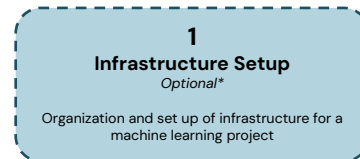
© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Infrastructure setup for MLOps is typically handled by an architect or engineer, such as a Platform Architect or Machine Learning Architect, and is usually configured once per project or organization to simplify management and enable reuse across multiple solutions. Core components include the Unity Catalog Metastore for centralized metadata management and governance, Databricks Workspaces to maintain environment integrity, and a command line environment for executing scripts and managing deployments. At the project level, the MLOps Stacks Project automates infrastructure creation, providing a standardized framework for pipelines, feature tables, and release workflows. Each project also maintains its own Git repository to support collaboration, code reviews, and version tracking of the codebase.

Infrastructure Setup

Getting set up for a machine learning project

- **Who:** Architect or Engineer
- **How often:** Set up once
- **What:**
 - A Unity Catalog metastore
 - One or three Databricks workspaces
 - Three data catalogs: dev, staging, prod
 - Command line environment
 - *MLOps Stacks* project (per project)
 - Git repository (per project)
- **How:**
 - Manual setup/Terraform
 - MLOps Stacks



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

- **How to Set Up:**
 - Manual = can be time-consuming and error-prone
 - or Terraform for automation and scalability.
 - **MLOps Stacks:**
 - MLOps Stacks is a tool provided by Databricks that automates the creation of infrastructure for an ML project workflow

Terraform is a general-purpose infrastructure-as-code tool that can be used with various cloud providers, while MLOps Stacks is specifically designed for MLOps workflows on Databricks.

Infrastructure Setup Tasks

Using our **DataOps** + **DevOps** + **ModelOps** mental model

- Make Infrastructure Decisions:
 - Number of **workspaces**
 - Use existing vs. new infrastructure
 - Select **CI/CD** tooling
- Create Databricks Environment:
 - **Unity Catalog Metastore**
 - 1 or 3 workspaces
 - **Unity Catalogs** (dev, test, staging, prod)
 - **Service principal permissions**
- Optimize for Efficiency:
 - **Project templates** with guardrails and best practices configured.
 - **Git repository** creation and connection
 - Configure Databricks **CLI and IDE**
 - *e.g., VS Code extension*
- Additional Considerations:
 - **Monitoring and logging**
 - **Backup and recovery**
 - Security best practices
 - Network configuration



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

- **Make Infrastructure Decisions:**
 - Determine the number of workspaces needed.
 - Decide whether to use existing infrastructure or create new infrastructure.
 - Select the CI/CD tooling to be used.
- **Create Databricks Environments:**
 - Set up Unity Catalog Metastore.
 - Configure 1 or 3 workspaces.
 - Establish Unity Catalog catalogs (development, test, staging, production).
 - Create a service principal with appropriate permissions.
 - A service principal is an identity that you create in Databricks for use with automated tools, jobs, and applications. Service principals give automated tools and scripts API-only access to Databricks resources, providing greater security than using users or groups.

Infrastructure Setup Tasks

Using our **DataOps** + **DevOps** + **ModelOps** mental model

- Make Infrastructure Decisions:
 - Number of **workspaces**
 - Use existing vs. new infrastructure
 - Select **CI/CD** tooling
- Create Databricks Environment:
 - **Unity Catalog Metastore**
 - 1 or 3 workspaces
 - **Unity Catalogs** (dev, test, staging, prod)
 - **Service principal permissions**
- Optimize for Efficiency:
 - **Project templates** with guardrails and best practices configured.
 - **Git repository** creation and connection
 - Configure Databricks **CLI and IDE**
 - *e.g., VS Code extension*
- Additional Considerations:
 - **Monitoring and logging**
 - **Backup and recovery**
 - Security best practices
 - Network configuration

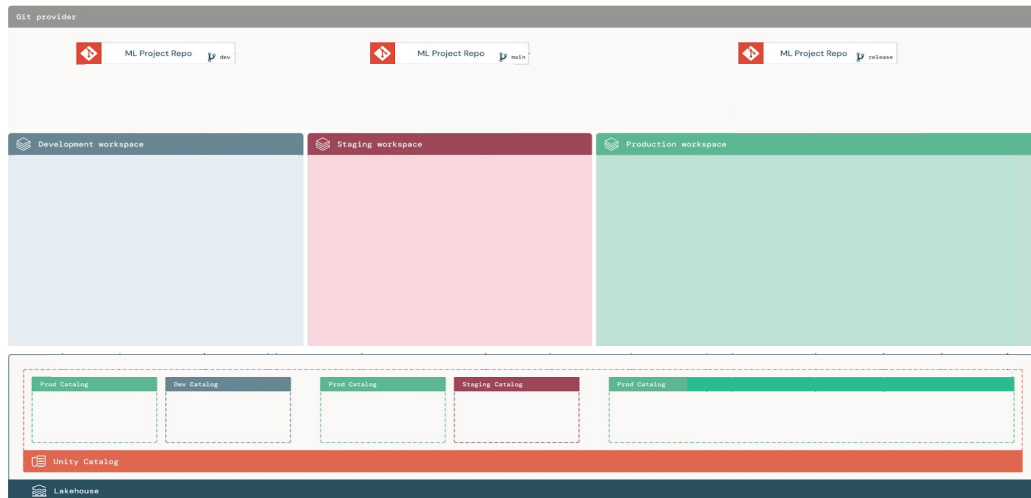


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

- **Make Infrastructure Decisions:**
 - Determine the number of workspaces needed.
 - Decide whether to use existing infrastructure or create new infrastructure.
 - Select the CI/CD tooling to be used.
- **Create Databricks Environments:**
 - Set up Unity Catalog Metastore.
 - Configure 1 or 3 workspaces.
 - Establish Unity Catalog catalogs (development, test, staging, production).
 - Create a service principal with appropriate permissions.
 - A service principal is an identity that you create in Databricks for use with automated tools, jobs, and applications. Service principals give automated tools and scripts API-only access to Databricks resources, providing greater security than using users or groups.

Deep Dive: Infrastructure

A closer look at the Infrastructure stage



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

W

Here we can take a closer look at the infrastructure stage.

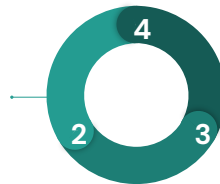
Development

Developing a machine learning project

- **Who:** Data Scientist
- **How often:**
 - Initial solution development
 - Solution updates
- **What:**
 - EDA
 - ML Development
 - ML Validation and Deployment
 - Monitoring solution
- **How:**
 - Develop ML pipelines within project architecture by editing an MLOps Stacks project

Development

Developing the EDA and ML Pipelines of the ML solution.



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

In MLOps, the data scientist plays a key role in developing and updating machine learning solutions to solve specific business problems. Their work includes initial solution development to establish the foundation, followed by iterative updates driven by analysis and feedback, with the frequency depending on business needs. Core activities involve exploratory data analysis to identify patterns and relationships, ML development through feature engineering, model selection, hyperparameter tuning, and training, as well as validation and deployment to ensure robustness and accuracy. Data scientists are also responsible for monitoring deployed models to track performance and impact over time. MLOps Stacks provides a framework to support these activities, allowing data scientists to configure workflows for data ingestion, preprocessing, training, evaluation, and deployment within a standardized project architecture.

Development Tasks

Using our **DataOps** + **DevOps** + **ModelOps** mental model

- Make Changes to Code:
 - Add and update **code**
 - Use **project templates** (DAB, MLOps Stacks, etc.)
 - Write **notebooks** and **scripts**
 - Create queries and alerts
 - Commit and pull code changes
- Validate Data and Code:
 - Ensure correct setup before deployment
 - Validate using **DLT**, **Asset Bundles**, etc.
 - Confirm **data format compliance**
 - Implement **data quality checks**
 - Review for security and compliance
- Deploy Solution:
 - Deploy all **jobs** to ensure successful execution
 - Use **CLI**, **scripts**, and **automation** for consistent deployments across environments
 - Establish **rollback strategies**
- Additional Considerations:
 - Implement **version control**
 - Optimize **deployment workflow**
 - Ensure scalability and performance **tuning**
 - Set up automated **notifications**

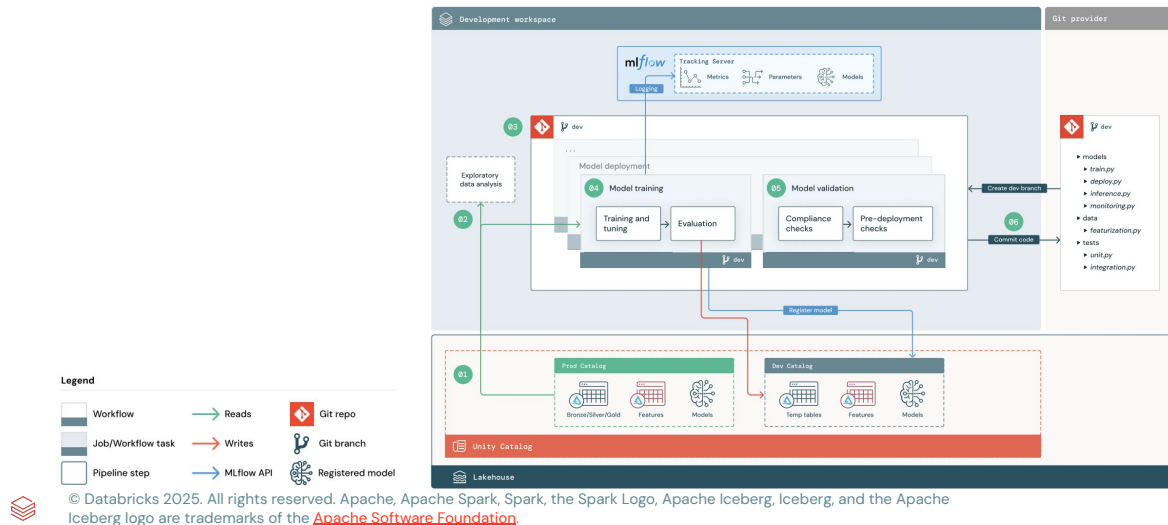


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://apache.org/).

In MLOps workflows, code is added or updated using project templates like DABs or MLOps Stacks, with notebooks, scripts, queries, and alerts managed through commits and pulls. Before deployment, data and code are validated using tools such as DLT and Asset Bundles, ensuring format compliance, data quality, and security standards. Solutions are deployed across environments with CLI, scripts, and automation, supported by rollback strategies for reliability. Additional best practices include maintaining version control, optimizing deployment workflows, ensuring scalability and performance, and enabling automated notifications.

Deep Dive: Development

A closer look at the Development stage



The development stage focuses on creating and refining machine learning models through exploratory data analysis, model training, evaluation, and initial code development. Data exploration leverages unified data management for secure access, followed by training, tuning, and validating models, with artifacts and metrics stored in the Development Catalog. Code for deployment, testing, and monitoring is developed and committed to Git for version control. Key tools include MLflow for experiment tracking and workflow management, Git for code management, the Databricks Unified Data Catalog for governance, and the Lakehouse architecture for seamless data integration and management.

Staging

Testing a machine learning project

- **Who:** ML Engineer
- **How often:**
 - Set up and run every time a change is made in Development
 - Run every time a model is refreshed
- **What:**
 - Project merge
 - Project code testing
- **How:**
 - Centralized Git repository
 - Automated CI/CD infrastructure tools



Staging

Establishing the automated testing setup for the ML solution.



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Instructor Notes:

- **Key Role:** ML Engineer: Primarily responsible for setting up and overseeing the testing framework that ensures ML models are ready for production.
- **Frequency of Activities:**
 - **Setup and Testing:** Conducted every time a change is made in the Development stage or whenever a model is refreshed.
 - **Ensures that all updates and model refreshes undergo thorough testing** before proceeding to the next stage.
- **Core Activities in Staging:**
 - **Project Merge:** Integrating developments from various branches to prepare for unified testing.
 - **Project Code Testing:** Extensive testing including unit tests, integration tests, and potentially regression tests.
- **Methodology:**
 - **Centralized Git Repository:** Utilizes a central repository for all code changes, facilitating version control and traceability.
 - **Automated CI/CD Infrastructure Tools:** Implements tools like GitHub Actions to automate the testing and integration processes, ensuring consistency and efficiency.

Staging Tasks

Using our **DataOps** + **DevOps** + **ModelOps** mental model

- Review CI/CD Workflows and Tests:
 - Examine existing **CI/CD** workflows
 - Add/change **tests** as needed
 - Ensure workflow alignment with project
- Create Pull Request to Run Tests:
 - Establish a **trigger**
 - e.g., pull request to merge dev branch into main branch
 - **Run Tests:**
 - Check for conflicts
 - Validate CI/CD setup
 - Validate the project
 - Unit, integration, and stress tests
- Analyze Test Results
 - If pass all tests and get approved,
 - merge into the main branch
 - If tests fail,
 - return to the development stage
 - Review test **reports and logs** for insights
- Additional Considerations:
 - Implement automated notifications for test results
 - Monitor staging environment performance
 - Ensure **data integrity** and consistency during staging

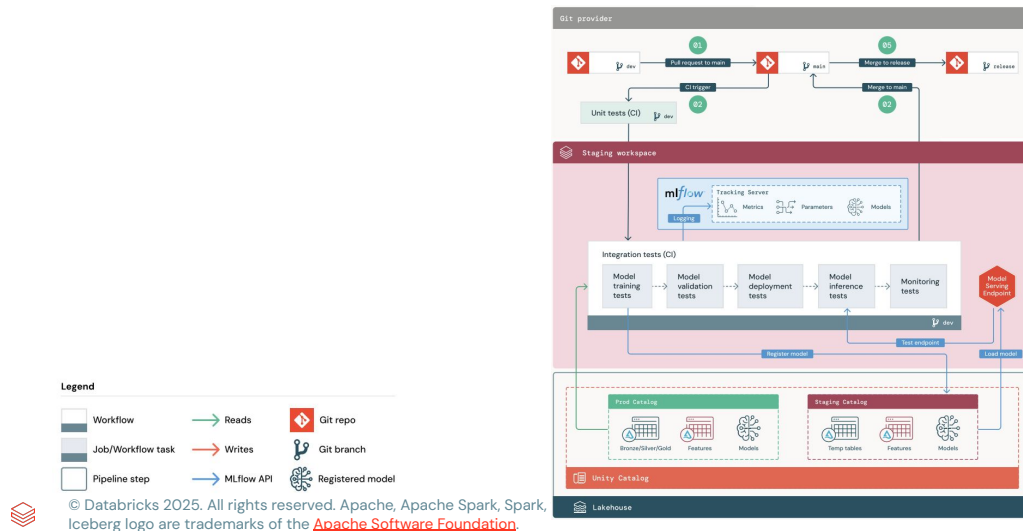


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

- Review existing CI/CD workflows and update/add tests as needed to ensure alignment with project requirements.
- Set up a trigger (e.g., pull request from dev to main) to run tests.
- Execute unit, integration, and stress tests while checking for conflicts, CI/CD validity, and overall project integrity.
- Analyze test results: if successful and approved, merge into main; if failed, return to development and review reports/logs for insights.
- Additional steps: enable automated notifications, monitor staging environment performance, and ensure data integrity and consistency.

Deep Dive: Staging

A closer look at the Staging stage

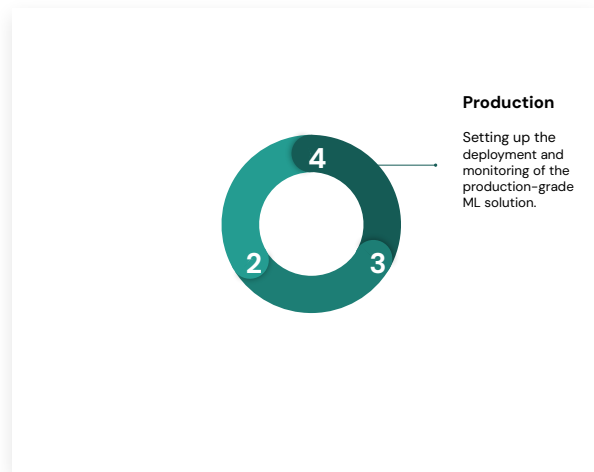


The staging phase focuses on code integration and rigorous testing, beginning with a Git pull request from development to main that triggers automated unit, integration, training, validation, deployment, inference, and monitoring tests. This process ensures models and code are thoroughly validated before reaching production, reducing risks and aligning with data governance standards. Data and artifacts are managed in a staging catalog for temporary writes during testing, kept strictly separate from production with enforced permissions to protect system integrity.

Production

Deploying and monitoring a machine learning project

- **Who:** ML Engineer
- **How often:**
 - When changes are made and tests are passed
 - When the model needs refreshed
- **What:**
 - Automated run/deployment of solution
 - Monitoring of solution
- **How:**
 - Centralized Git repository
 - MLOps Stacks project



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

- **Key Role:** ML Engineer
 - Charged with the deployment and ongoing monitoring of machine learning solutions in the production environment.
- **Frequency of Activities:**
 - **Upon Passing Tests:** Activities are triggered when changes or updates have passed all tests in the staging environment.
 - **Model Refresh:** When existing models are refreshed to incorporate new data or improvements.
- **Core Activities in Production:**
 - **Automated Run/Deployment of Solution:** deployment process is efficient and automated, reducing the need for manual intervention.
 - **Monitoring of Solution:** Continuously observing the performance and impact of deployed model
- **Methodology:**
 - **Centralized Git Repository:** Utilizes a centralized repository for managing all production code, ensuring consistency and control over version management.
 - **MLOps Stacks Project:** Leverages the structured framework provided by MLOps Stacks to maintain and streamline deployment and monitoring processes.

Production Tasks

Using our **DataOps** + **DevOps** + **ModelOps** mental model

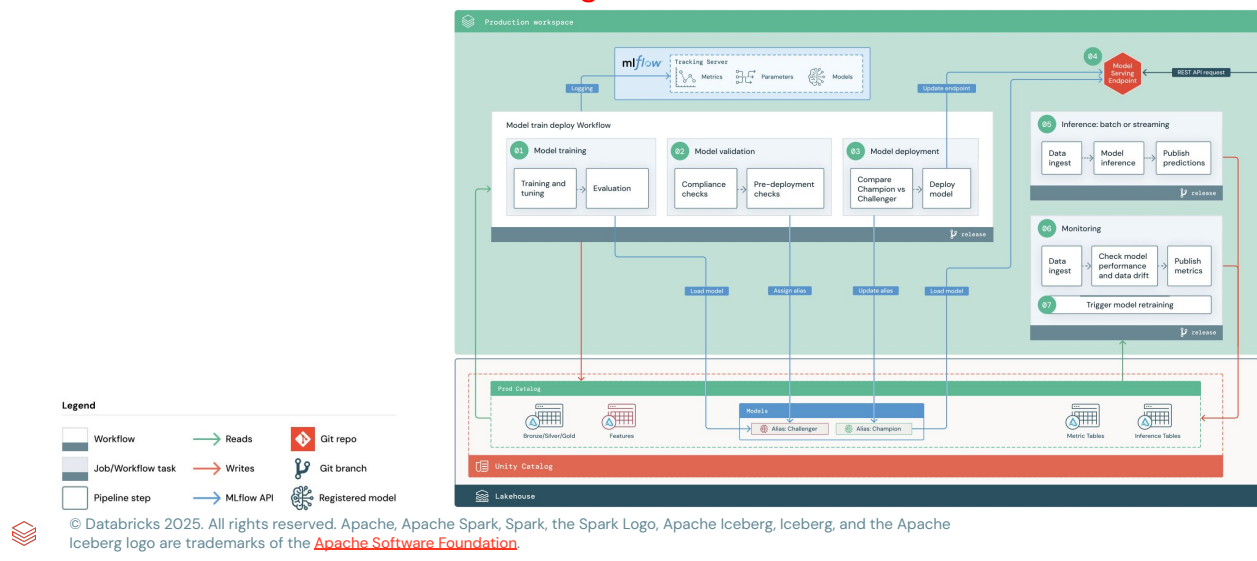
- Create/Merge to Release Branch:
 - Set up release branch
 - Include deployment **triggers**
 - Merge updates into release branch
- Deploy Solution:
 - Automatic **deployment triggered** by release branch updates
 - Deploy components:
 - **Project code**
 - **Data, Model**, and **Monitoring** workflows
 - **Compute resources**
- Monitor
 - Track **system performance**
 - Monitor **deployment health**
 - Real-time **alerting** and notifications
 - Log analysis for error detection
 - **Data** and **model** drift detection
- Additional Considerations:
 - Optimize **resource utilization**
 - Ensure compliance with security policies
 - Conduct post-deployment **reviews**
 - Implement **incident response** protocols
 - Set up **automated retraining pipelines**



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

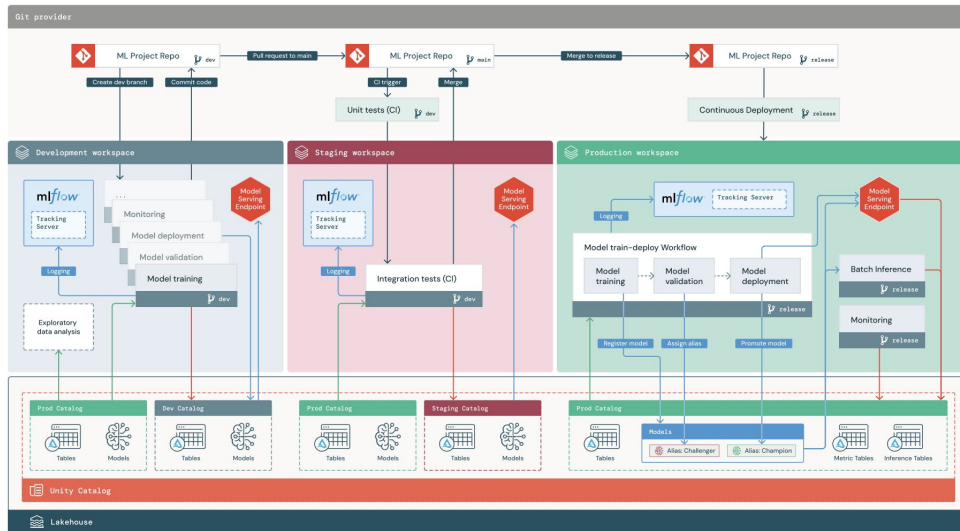
- Create a release branch with deployment triggers and merge updates into it.
- Deploy automatically on release branch updates, covering project code, data, models, monitoring workflows, and compute resources.
- Monitor system performance, deployment health, real-time alerts, log analysis, and detect data/model drift.
- Additional steps: optimize resource usage, ensure security compliance, conduct post-deployment reviews, implement incident response protocols, and set up automated retraining pipelines.

A closer look at the Production stage



The production stage is the final phase of the MLOps lifecycle, where tested code is deployed from the main Git branch and models are managed through a complete lifecycle of retraining, evaluation, registration, validation, comparison, and deployment. Data and artifacts are fully integrated with the production catalog, ensuring secure, compliant handling. Automated workflows streamline processes, while continuous monitoring and feedback loops detect performance issues or data drift, triggering alerts and retraining to maintain model reliability and accuracy in real-world operations.

Complete MLOps Architecture



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

- **Benefits of This Architecture:**

- **Holistic Management:** Provides a clear visualization and management strategy for every aspect of ML operations.
- **Scalability and Flexibility:** Supports scaling of ML operations with robust data infrastructure and controlled code deployment.
- **Enhanced Collaboration:** Facilitates team collaboration across different stages of ML project development by providing clear workflow definitions and tool integrations.



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Thank you for completing this lesson and continuing your journey to develop your skills with us.